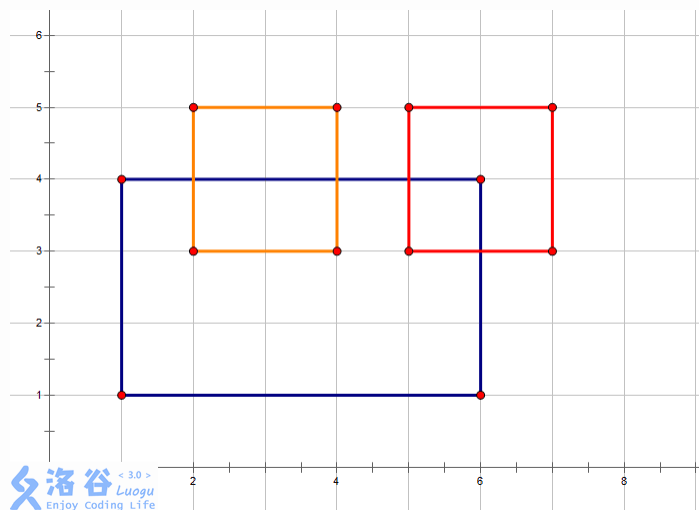
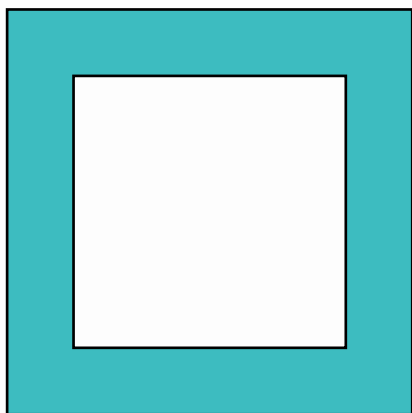


矩形周长之和

上题问面积之和是多少，那么如果问你周长之和呢？周长也就是围成的形状的表面的边长之和（图中黑色线条为周长）。



矩形周长之和

首先，矩形的周长分为长和宽，如果我们按照前面求面积的方法来做，宽度就不太好求，所以我们可以把矩形的长和宽分开来求解，即先求水平的，再求垂直的。

在考虑的时候，我们发现要分上底面和下底面考虑。

对于下底面的边 $[s, t]$ ，先查询 $[s, t]$ 之间被覆盖的长度 x ，然后 $t-s-x$ 就是新增的

对于上底面的边 $[s, t]$ ，先删除 $[s, t]$ 对应的边，然后再查询 $[s, t]$ 之间被覆盖的长度 x ，然后 $t-s-x$ 就是。

线段树的用途

线段树是一种暴力的数据结构，他的用途十分广泛，它主要用于求解区间修改，区间查询这一类的问题，只要一个大区间可以分解成多个小区间，或者两个小区间的信息可以合并到大区间去，一般都可以用线段树来维护。

但是线段树是不是万能的呢？

并不是，因为线段树的建树是开始就建好的，线段树代表的区间在建树那一刻就固定了，所以线段树是不支持区间变化的查询的，比如点删除，点添加，区间删除，区间添加等等，一旦涉及区间变化，那么整个线段树的形态就会发生改变，那么线段树就需要重建，效率就低了。一般涉及区间变化都是用平衡树实现。

线段树的用途

但是对于一些特殊的区间修改，比如区间删除，或者单点删除，或者部分单点添加，实际上线段树还是可以做的，以前的线段树，对于区间 $[l, r]$ ，那么实际存在的点的个数就是 $r-l+1$ 个，但是遇到这种点删除的情况。如果我们要让区间仍然不变，那么我们就要再维护当前区间真实节点数量，比如删除 i 号节点，如果 i 号节点在左边，那么左边真实儿子数量--，如果 i 号节点在右边，那么右边真实儿子数量--。如果要查询区间 $[l, r]$ ，主要是去看当前节点的左右子树的真实儿子。如果 $r \leq$ 左儿子的真实儿子数量，那么就去左区间查找，如果 $l >$ 右儿子的真实儿子数量，那么就去右区间查找。

动态最值

题目描述

[展开](#)

有一个包含 n 个元素的数组，要求实现以下操作：

- `DELETE k`：删除位置 k 上的数。右边的数往左移一个位置。
- `QUERY i j`：查询位置 $i \sim j$ 上所有数的最小值和最大值。

例如有 10 个元素：

位置	1	2	3	4	5	6	7	8	9	10
元素	1	5	2	6	7	4	9	3	1	5

`QUERY 2 8` 的结果为 `2 9`。依次执行 `DELETE 3` 和 `DELETE 6`（注意这时删除的是原始数组的元素 7）后数组变为：

位置	1	2	3	4	5	6	7	8
元素	1	5	6	7	4	3	1	5

`QUERY 2 8` 的结果为 `1 7`。

输入格式

第一行包含两个数 n, m ，表示原始数组的元素个数和操作的个数。

第二行包括 n 个数，表示原始数组。

以下 m 行，每行格式为 `1 k` 或者 `2 i j`，其中第一个数为 1 表示删除操作，为 2 表示询问操作。

输出格式

对每个询问操作输出一行，包括两个数，表示该范围内的最小值和最大值。



动态最值

第一步：线段树建树

和常规线段树建树没区别，只是多记录一个区间真实节点数量，最开始的时候所有节点都存在，所以节点数量就是区间长度。



动态最值

第二步：线段树单点删除

假设我要求删除当前序列中第 k 个节点，根当前节点开始，不再是和 mid 做比较，因为 mid 和区间真实节点数量没有任何关系，我们判断在左区间还是在右区间，那么比较的就是左右区间真实节点数量，所以用 k 和 $tree[id*2].son$ 做比较，如果 $k < tree[id*2].son$ ，说明查找的位置在左区间，否则查找的位置在右区间。但是我们要注意那么我们应该在右区间查找第几个节点呢？还是第 k 个？并不是，因为右子树的节点是不包含左子树的，所以应该在右子树里面查找第 $k - tree[id*2].son$ 个节点。这是最容易错的地方， son 只是表示当前节点的儿子数量，也就是没有算左子树的儿子个数，所以要减去。

动态最值

```
void update(int id,int k)
{
    if(tree[id].l==tree[id].r)//既然是单点修改, 所以是修改叶子节点
    {
        tree[id].son=0;//真实儿子个数为0
        tree[id].Max=-inf;//存在负数
        tree[id].Min=inf;
        return ;
    }
    if(k<=tree[id*2].son)//和当前节点左儿子的节点数作比较
    update(id*2,k);
    else
    update(id*2+1,k-tree[id*2].son);//注意要减去左儿子数量
    tree[id].Max=max(tree[id*2].Max,tree[id*2+1].Max);
    tree[id].Min=min(tree[id*2].Min,tree[id*2+1].Min);
    tree[id].son=tree[id*2].son+tree[id*2+1].son;
}
```


动态最值

第三步：线段树区间查询

对于当前节点，如果我要查询的区间是 $[l, r]$ ，那么如果 $r \leq \text{tree}[\text{id} * 2].\text{son}$ ，那么说明这一段都在左区间，那么去左区间查找，如果 $l > \text{tree}[\text{id} * 2].\text{son}$ ，那么我们就去右区间去查找这一段，但是右子树同样不包含左子树的节点数，所以 l 和 r 都要同时减去

$\text{tree}[\text{id} * 2].\text{son}$ ，再去右区间查找，不能直接查找区间 $[l, r]$ 。如果两个区间都有一部分，那么左区间的能查找的最后一个数就是 $\text{tree}[\text{id} * 2].\text{son}$ ，在右区间查找的第一个数就是 $\text{tree}[\text{id} * 2 + 1].\text{son}$

那么什么时候查找结束呢？ $r - l + 1 = \text{区间长度}$ 或者 $l = 1$ ， $r = \text{tree}[\text{id}].\text{son}$ 的时候，这两者只要一个满足，另一个也一定满足

线段树

```
node query(int id,int l,int r)
{
    //if(l==1&&r==tree[id].son)
    if(r-l+1==tree[id].son)//如果找到一个完整区间
        return tree[id];
    if(r<=tree[id*2].son)//都在左区间
        return query(id*2,l,r);
    else if(l>tree[id*2].son)//都在右区间
        return query(id*2+1,l-tree[id*2].son,r-tree[id*2].son);
    else
    {
        node t1=query(id*2,l,tree[id*2].son);
        node t2=query(id*2+1,1,r-tree[id*2].son);
        node tmp;
        tmp.Max=max(t1.Max,t2.Max);
        tmp.Min=min(t1.Min,t2.Min);
        return tmp;
    }
}
```

线段树

前面只是涉及到线段树单点删除的操作，区间删除的做法和单点删除是一样的，参照区间查询。

但是如果是区间增加呢？先以简单的单点增加为例，如果是随意加点，那么线段树是做不了的，比如在每次增加操作在第 i 个数之后添加一个数，后面的数后移。但是有一种特殊的增加他是可以的，比如每次都在末尾添加一位数，或者在开头添加一个数，或者在固定位置添加一个数。

那么对于这一类的题应该如何做呢？

线段树

对于删除操作我们知道怎么做了，但是对于增加操作我们怎么处理呢？实际上我们可以看做删除操作，这里介绍一种办法——留坑法，先看下有多少次增加操作，那么最开始建线段树的时候就以 $n+k$ 为区间建立线段树，而不是以 $1—n$ 为区间建树，但是最开始把区间 $[n+1, n+k]$ 的所有节点设置为失效节点。如果有插入操作，那么我就把该位置的点变为有效节点。

这就是线段树的一种失效节点，有效节点的办法，如果有添加操作，就把失效节点变为有效节点，如果有删除节点，就把有效节点变为失效节点，每次只统计有效节点的答案。

线段树

给定一个长度为 n 的序列，给定两个操作

- 1 x : 在序列末尾添加一个数 x
- 2 x : 删除当前序列序列第 x 个数
- 2 $l\ r$: 查询区间 $[l, r]$ 的最大值

线段树

题目描述

[展开](#)

Sylvia 是一个热爱学习的女孩子。

前段时间, Sylvia 参加了学校的军训。众所周知, 军训的时候需要站方阵。

Sylvia 所在的方阵中有 $n \times m$ 名学生, 方阵的行数为 n , 列数为 m 。

为了便于管理, 教官在训练开始时, 按照从前到后, 从左到右的顺序给方阵中的学生从 1 到 $n \times m$ 编上了号码 (参见后面的样例)。即: 初始时, 第 i 行第 j 列的学生的编号是 $(i - 1) \times m + j$ 。

然而在练习方阵的时候, 经常会有学生因为各种各样的事情需要离队。在一天中, 一共发生了 q 件这样的离队事件。每一次离队事件可以用数对 (x, y) ($1 \leq x \leq n, 1 \leq y \leq m$) 描述, 表示第 x 行第 y 列的学生离队。

在有学生离队后, 队伍中出现了一个空位。为了队伍的整齐, 教官会依次下达这样的两条指令:

1. 向左看齐。这时第一列保持不动, 所有学生向左填补空缺。不难发现在这条指令之后, 空位在第 x 行第 m 列。
2. 向前看齐。这时第一行保持不动, 所有学生向前填补空缺。不难发现在这条指令之后, 空位在第 n 行第 m 列。

教官规定不能有两个或更多学生同时离队。即在前一个离队的学生归队之后, 下一个学生才能离队。因此在每一个离队的学生要归队时, 队伍中有且仅有第 n 行第 m 列一个空位, 这时这个学生会自然地填补到这个位置。

因为站方阵真的很无聊, 所以 Sylvia 想要计算每一次离队事件中, 离队的同学的编号是多少。

注意: 每一个同学的编号不会随着离队事件的发生而改变, 在发生离队事件后方阵中同学的编号可能是乱序的。

$$n, m, q \leq 3 \times 10^5$$



线段树

该题你会发现数组开不下，而且是真实存在这么多个节点，所以也不能离散化。因为涉及到单点修改，区间变化，所以需要用数据结构来维护，但是如果你对每一行开一棵线段树或者平衡树，时间复杂度肯定受不了，那应该怎么办呢？



线段树

实际上线段树还有一种动态开点的办法，一般的线段树都是在最开始的时候就把线段树每个节点建好，然后再去修改查询，这里介绍一种最开始不建线段树的做法，最开始只有一个节点，如果有一次修改操作。比如单点修改，那么我就只对需要查询到的点新增节点，其他访问不到的点我不用去建节点，那么每次修改的时间辅助就是 $\log n$ 。同样访问也是一样，我要去查询这个区间，我只把需要用到的点建出来，其他点不管，最坏时间复杂为 $\log n^2$ 。和线段树点的区间没有关系。

但是这种线段树只能适用于知道最开始线段树所有节点信息的题

线段树

一：最开始

①

$[1, 10]$

二：修改编号为k的节点，权值为x

比如 3 5

①

$[1, 10]$

②

$[1, 5]$

③

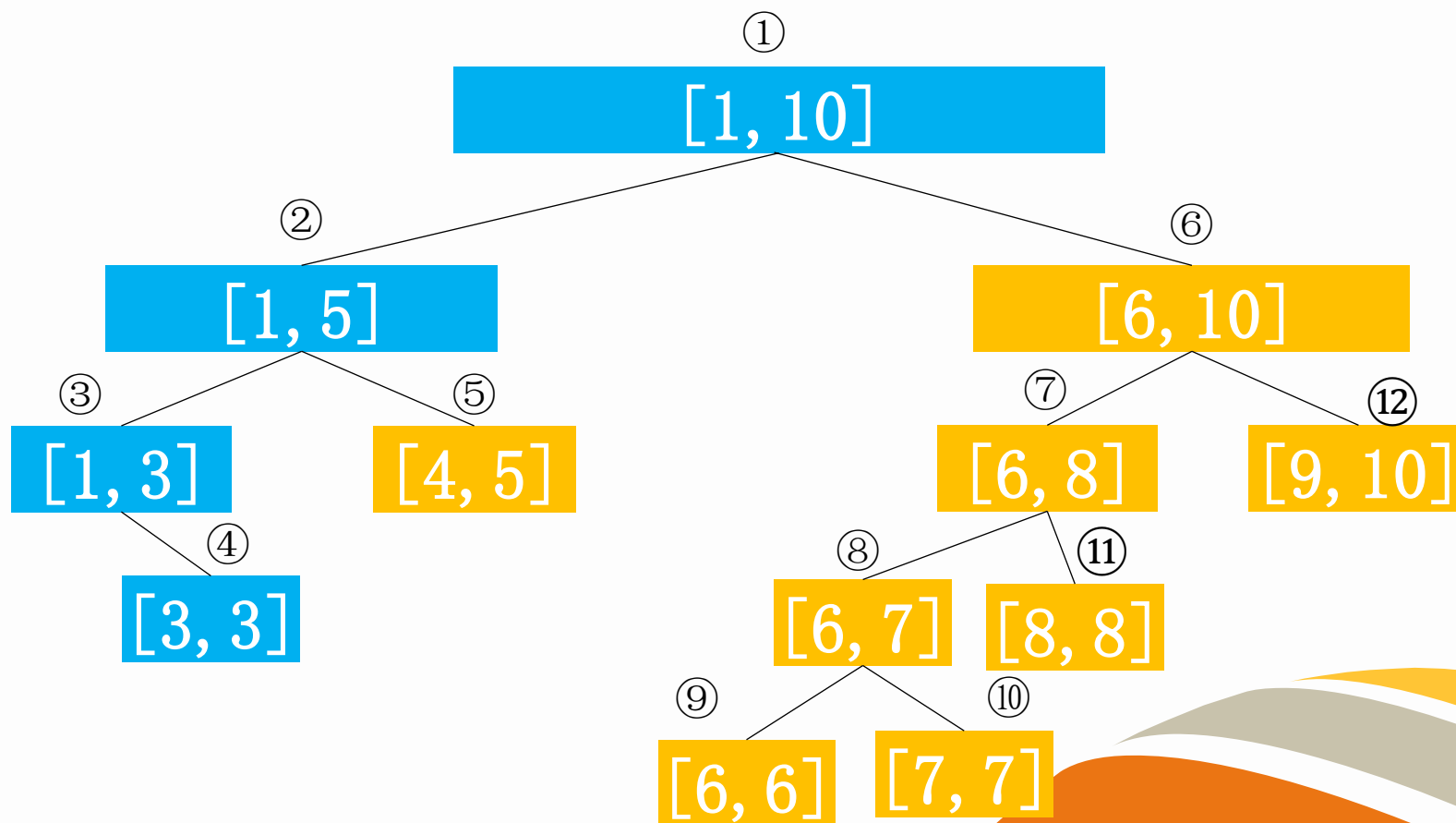
$[1, 3]$

④

$[3, 3]$

线段树

三：查询区间 $[2, 7]$ 的最大值



线段树

我们把这道题简化一下，假设有一个二维矩阵，最开始的时候初始值都是0，每次修改一个行区间，每次再询问一个行区间最大值。

$n, m, q \leq 3e5$

线段树

第一步：线段树建树

```
int build(int id,int l,int r)
{
    ++k;//动态开点
    tree[id].l=l;
    tree[id].r=r;
    return k;//最开始只有一个根节点
}
for(int i=1;i<=n;i++)
    root[i]=build(1,1,m);//建立n棵行线段树
```

线段树

第二步：线段树区间修改

```
void update(int id,int l,int r,int x)
{
    if(tree[id].l==l&&tree[id].r==r)
    {
        ...
        return ;
    }
    int mid=(tree[id].l+tree[id].r)/2;
    if(!tree[id*2])//如果左二子不存在
    tree[id].ls=build(l,mid);
    if(!tree[id*2+1])//如果右儿子不存在
    tree[id].rs=bbuild(mid+1,r);
    //标记下传
    if(r<=mid)
    update(id*2,l,r,x);
    else if(l>mid)
    update(id*2+1,l,r,x);
    else
    {
        update(id*2,l,mid,x);
        update(id*2+1,mid+1,r,x);
    }
    tree[id].Max=max(tree[tree[id].ls].Max,tree[tree[id].rs].Max);
}
update(root[i],l,r,x)//修改第i行的区间[l,r]增加一个值x
```

线段树

第三步：线段树区间查询

```
int query(int id,int l,int r)
{
    if(tree[id].l==l&&tree[id].r==r)
    {
        return tree[id].Max;
        return ;
    }
    int mid=(tree[id].l+tree[id].r)/2;
    if(!tree[id*2])//如果左二子不存在
        tree[id].ls=build(l,mid);
    if(!tree[id*2+1])//如果右儿子不存在
        tree[id].rs=bbuild(mid+1,r);
    //标记下传
    if(r<=mid)
        return query(id*2,l,r,x);
    else if(l>mid)
        return query(id*2+1,l,r,x);
    else
        return max(query(id*2,l,mid,x),query(id*2+1,mid+1,r,x));
}
query(root[i],l,r);
```